Recall First, Then Let the Model Choose: Gemma 4 as a Bug-Localization Selector, Tested on Post-Cutoff Issues

Hussam — Independent Researcher

Kaggle · Google Gemma 4 Developer Agent Paper Track · November 2026

Abstract

Before a coding agent can fix a bug it must find the file to edit. We compare two ways of spending Gemma-4-12B-it on this step. As a **scorer** — ranking code by how much the issue lowers the model's cross-entropy on it (NAMI) — the model is weaker than BM25. As a **selector** — shown the issue and a shuffled list of candidate paths, asked to name the likeliest files — it is strong: on the *same* BM25 candidates on 57 SWE-bench Lite instances it places the gold file in the top-5 on 36 instances against NAMI's 15, and on a BM25 ∪ dense pool on 84%, against 65% for dense retrieval and 75% for an oracle choosing the better retriever per instance. Because SWE-bench predates Gemma 4, we then test what the model brings. Removing or swapping the issue drops top-5 to 30% and 19%; on 120 SWE-bench-Live issues created after the model's cutoff, with the pipeline frozen, the selector beats dense retrieval at top-1 (61% vs 38%, significant) and leads at top-5 (85% vs 72%, not significant after correction). One limit remains: on new issues that never name the gold file it only ties dense retrieval.

1. Introduction

A file that never reaches an agent's context window cannot be fixed, and most repository-level agents localize in two stages: retrieve candidates, then narrow them down. We began with an information-theoretic bet about the second stage: if an issue carries information about the faulty code, prefixing it should lower a model's cross-entropy on exactly that code, giving a training-free, generation-free relevance score. The bet lost, and its controls led to three findings:

1. **On identical candidates, choosing beats scoring.** One listwise call puts the gold file in the top-5 on 36/57 instances where likelihood scoring of the same 25 files manages 15 (§4.1).
2. **Pool recall is the next bottleneck.** BM25's top-25 contains the gold file on 63% of instances; adding a 90 MB embedder's top-25 raises this to 89%, and the selector converts that into 84% top-5 (§4.2–4.3).
3. **The selector's skill survives a post-cutoff test.** Controls and 120 issues created after its training cutoff separate reading from recall, and show where the gain is and is not (§4.4).

2. Related work

Likelihood for retrieval and defects. Query-likelihood ranking with open LLMs scores $P(\,\text{query} \mid \text{doc}\,)$ [1]; null-prompt calibration comes from multiple-choice debiasing [2]; context–question PMI predicts RAG accuracy [3]. Unconditioned surprisal as a defect signal goes back to [4, 5]. We score the candidate given the issue, null-calibrated, and find it behaves like query likelihood on code. **LLM rerankers and selectors.** Listwise reranking [6] has been adapted to issue localization by LLM-BL [7], CodeRankLLM [8] and SweRank [9], the latter two fine-tuned; Agentless [10] narrows hierarchically and LocAgent [11] searches a code graph. Our selector is a zero-shot member of this family; what we add is the same-pool comparison against likelihood scoring, the pool-ceiling decomposition, and a memorization audit. **Memorization.** Models

identify SWE-bench file paths from the issue alone far more often than on other repositories [12, 13]. Loc-Bench [11] starts in October 2024, inside Gemma 4's training window (January 2025 cutoff [16]), so we use SWE-bench-Live [17] issues created from April 2025.

3. Methods

Likelihood scoring (NAMI). For issue I and an AST chunk $C = (t_1..t_N)$, a frozen causal LM gives $H(C \mid X) = -\frac{1}{N}\sum_i \log_2 P(t_i \mid X, t_{<i})$. NAMI $= H(C \mid I_{null}) - H(C \mid I)$ — a pointwise, PMI-style contrast — with I_{null} a bug report from another, non-scientific repository matched to ±30% of I's length. Files take their best chunk's score. Controls: query likelihood (QLM), mean $\log_2 P(I \mid C)$; a no-null ablation ranking by $-H(C \mid I)$; reciprocal-rank fusion (RRF, k=60) with BM25. The scorer runs Gemma's backbone only and applies the LM head to 128 positions at a time, so the 262k-vocabulary logits are never held for a whole chunk; memory stays flat on one A100 (bf16).

Retrievers. Tree-sitter chunks (functions and classes; whole files ≤4,000 characters when nothing parses) over all Python files outside `tests/` and `docs/` at the base commit. BM25 (`rank_bm25` Okapi defaults, lower-cased whitespace tokens, chunks ≤6,000 characters) and `all-MiniLM-L6-v2` cosine similarity (default 256-token window), each max-aggregated to files.

Listwise selection. Pool = BM25 top-25 ∪ dense top-25 (28–49 files, mean 41.8), shuffled with a fixed per-instance seed so the model cannot copy retrieval order. Gemma receives the repository name, the issue (first 4,000 characters; hints are not used) and the numbered list — paths only, or paths plus up to 15 class/function names per file — through its default chat template as one user turn (no system prompt, no thinking flag), greedy, ≤80 new tokens (Appendix A). The first 10 distinct valid numbers form its top-10; all 114 Lite answers and 119/120 post-cutoff answers gave 10, and unnamed pool files are appended in BM25 order. A call costs about 1.1k prompt tokens and 4.6 s on an A100 (paths only) or 3.8k tokens and 7.6 s (with names).

Data and statistics. 61 SWE-bench Lite [14] instances stratified round-robin over 8 repositories; 57 completed likelihood scoring (4 ran out of memory and are excluded from every comparison): django 19, sympy 8, astropy 6, requests 6, matplotlib 5, pylint 5, scikit-learn 5, flask 3. Each has one gold file. An issue *names the file* if the gold file's base name without `.py` (e.g. `autoreload`) occurs in the issue text, case-insensitively, ignoring the stems `__init__`, `base`, `utils`, `core`. We report hits on identical instances with 95% bootstrap CIs (10,000 resamples) and exact two-sided McNemar tests on paired hits (+b/−c = instances only the first/second method gets right). Tests are Holm-corrected within six question families (§4.1; the in-pool reranker comparisons of our earlier analysis; §4.3; controls; probe; post-cutoff — 40 tests, all listed in the released statistics output); every result we call significant survives that correction, and we mark those that do not. *Protocol change:* the planned pool paired BM25 with CodeRankEmbed, which failed at its first encoding; the likelihood methods therefore rerank BM25's top-25 on every instance.

4. Results

4.1 On the same candidates, choosing beats scoring

ranking BM25's top-25 files (57 instances)	top-1	top-5	top-10
NAMI (per-token)	5%	26%	40%
NAMI total-bits / QLM / no-null ablation	11 / 9 / 5%	23 / 25 / 26%	47 / 46 / 42%
Random order of the 25 (expected)	3%	13%	25%
BM25's own order	28%	51%	56%
Gemma selects from the same 25	**47%**	**63%** [51,75]	63%

Gemma beats NAMI at top-5 (+21/−0) and top-1 (+24/−0, both $p<0.0001$) and BM25's order (top-1 +12/−1, $p=0.003$; top-5 +7/−0, $p=0.016$). On the 36 instances whose gold file is in this pool, Gemma's top-5 contains it on all 36; BM25's order on 29 (81%), NAMI's on 15 (42%). Likelihood scoring is therefore well above random but weaker than BM25 and adds nothing to it: NAMI and its no-null ablation both reach 15/57 top-5 hits (on different instances, +6/−6), so the null contrast adds nothing, and fusing NAMI into BM25 lowers BM25's top-5 from 29 to 21 hits (+0/−8, $p=0.008$).

An earlier study with *random* distractor files had made the signal look complementary to BM25; a padding bug (fixed) and realistic distractors removed that impression. Random files differ in generic "naturalness"; retrieved candidates already share the issue's vocabulary, so likelihood retrieval should be evaluated on first-stage pools.

4.2 Pool recall is the next bottleneck

On 21/57 instances the gold file is not in BM25's top-25 at all, and dense retrieval ranks 15 of those in its own top-25 (django-14155: BM25 rank 59, dense rank 1). **BM25-25 ∪ dense-25 contains the gold file on 51/57 instances (89%) instead of 36 (63%)** (Figure 1a). NAMI was not re-scored on the union pool (about 10 A100-hours), but chunks are scored independently of the other candidates, so a larger pool cannot raise the gold file's rank: NAMI's union top-5 is at most 15 + 15 = 30/57 (53%).

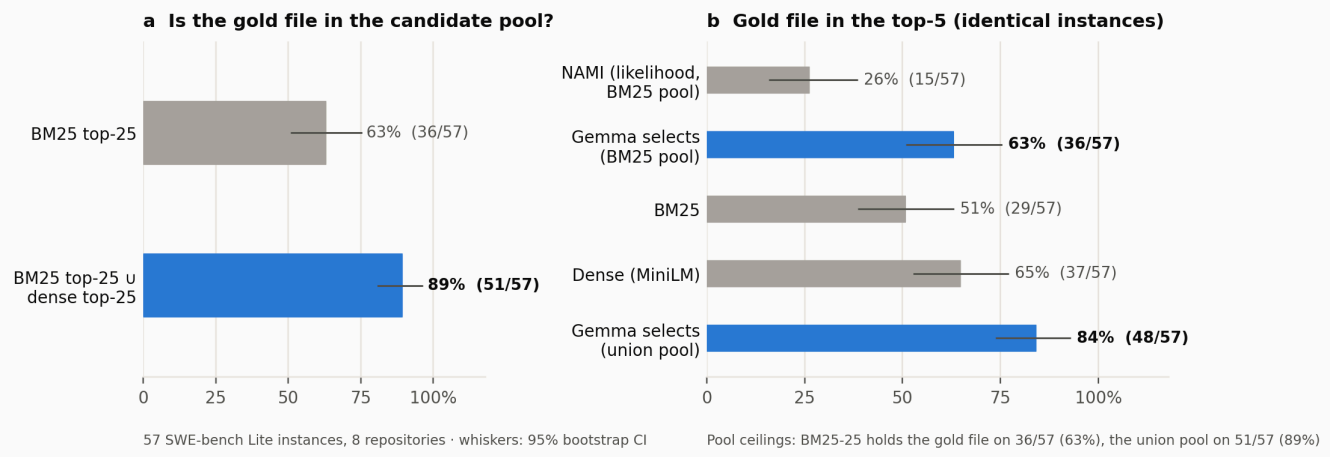


Figure 1. (a) Instances whose gold file is in the candidate pool. (b) Gold file in the top-5 on the same 57 instances. Whiskers: 95% bootstrap CIs.

4.3 A high-recall pool and a selector

union pool (identical 57 instances)	top-1	top-5	top-10	MRR
BM25	28%	51%	56%	0.38
Dense	44%	65%	79%	0.54
RRF(BM25, dense)	35%	58%	74%	0.46
Oracle: better of BM25/dense per instance	54%	75%	84%	—
Gemma, paths only	61% [49,74]	**84%** [74,93]	88%	0.72
Gemma, paths + names	**68%** [56,81]	**84%** [74,93]	88%	0.76

Gemma's top-5 beats dense (paths +13/−2, p=0.007; with names +12/−1, p=0.003) and BM25 (+20/−1, p<0.0001). At top-1 the gain is significant with names (+19/−5, p=0.007) but not with paths only (+18/−8, p=0.08); the two Gemma conditions do not differ (+5/−1, p=0.22). The pool already contains dense's top-25, so the gap is choice, not recall: of the 51 gold files the pool holds, Gemma's top-5 recovers 94%, dense's 73%. Two further shuffles of every candidate list give top-5 of 48 and 49 and top-1 of 35 and 36; 53/57 instances have the same top-5 outcome in all three orders. A call costs ~1.1k tokens and 4.6 s, against a median of 205 s of NAMI scoring per instance (323 s with QLM).

4.4 What does the model bring?

SWE-bench is public and predates Gemma 4, so a selector might succeed by recalling answers. Three tests separate reading the issue from recall.

Without the issue the selector collapses. With the same pools and shuffles, replacing the issue by a placeholder gives top-5 17/57 (30%), and substituting another issue from the same repository gives 11/57 (19%), against 48/57 with the real issue (+31/−0 and +37/−0). About 30% is what repository and path priors buy.

Issue-only, hard options. On all 57 instances we give the model only the issue and ten paths — the gold file and the nine highest-ranked non-gold files of BM25 and dense — and score other readers on the identical options (Figure 2). Gemma picks the gold file on 44 (77%), against 25 for dense and 16 for BM25 over the files' code, and 18 for matching path tokens against the issue text with ties broken by BM25 order (+27/−1). On the 23 issues that do not name the gold file, Gemma is right on 16, path matching on 1 (+15/−0, p=0.0001) and dense on 8 (+11/−3, p=0.057). Its letter choices are spread over all ten positions (one answer did not parse and counts as wrong). Lexical name matching does not explain the selector.

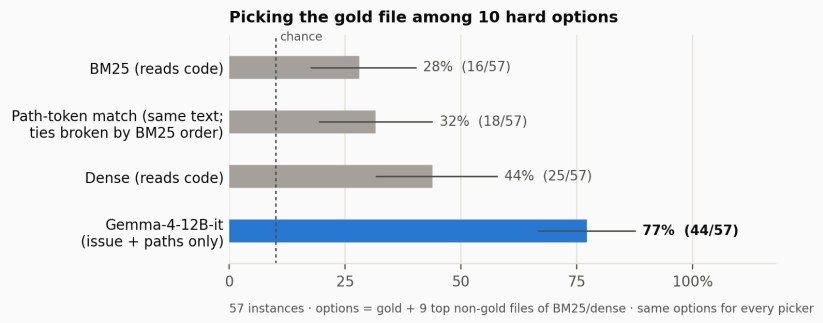


Figure 2. Issue-only 10-way choice with hard distractors on the 57 instances. Distractors are the top non-gold files of BM25 and dense, so those readers are disadvantaged by construction. Chance: 10%.

After the cutoff. We sampled 120 SWE-bench-Live issues created between April and August 2025 (at most 3 per repository, 84 repositories), three months past Gemma 4's January 2025 cutoff, and ran the frozen pipeline: same retrievers, pool recipe, seed rule and prompt, no re-tuning, nothing executed. Gold files are the non-test Python files changed by the fix (47 instances change several; a hit is any gold file). These repositories are smaller (median 127 Python files against 718), so the union pool holds a gold file on 118/120.

post-cutoff (120 instances)	top-1	top-5	top-10
BM25	33%	55%	69%
Dense	38%	72%	79%
Gemma selects	**61%** [53,69]	**85%** [78,91]	**95%**

Gemma beats dense at top-1 (+41/−14, p=0.0004) and on the 73 single-file instances (top-1 42 vs 23, p=0.003); at top-5 the gain (+27/−12, p=0.024) does not survive Holm correction. Gemma's top-5 advantage over dense is +13 points [+3, +22] (95% CI, resampling whole repositories), against +19 [+10, +34] on Lite: smaller, with overlapping intervals. Nor does the gain fade with distance from the cutoff — the reverse: in April–May, the months closest to it (51 issues), Gemma and dense are about even (top-5 73% vs 71%), and Gemma's lead appears from June on. The limit is in the name-free slice: on the 49 issues that never name a gold file, Gemma ties dense (top-5 36 vs 37, top-1 19 vs 19), and is only nominally ahead of BM25 (36 vs 24, p=0.03, not significant after correction). Our reading: the selector's advantage is real reading of the issue that transfers to unseen repositories; its edge over a good embedder comes mostly from issues that point at the right file, and some Lite inflation cannot be excluded.

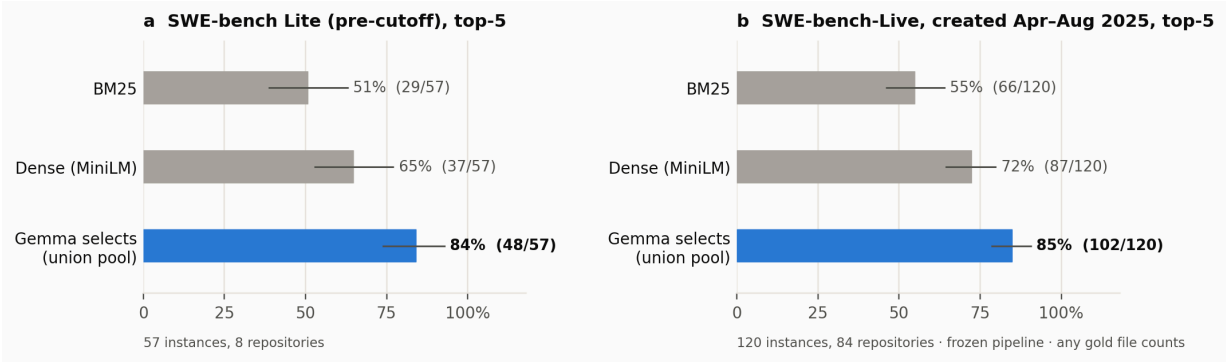


Figure 3. Top-5 before and after Gemma 4's training cutoff, same frozen pipeline. Whiskers: 95% bootstrap CIs.

4.5 Localization is necessary, not sufficient

In a single-turn patch demonstration on django (10 of 12 instances passed the gate: the official fix must pass the target tests that fail on the base commit), the gold file reached the 12B patch

writer 3, 6 and 8 times with BM25, dense and Gemma top-5 context, but 1, 0 and 1 patches resolved: patch synthesis is a separate constraint, which is why we study the first mile in isolation.

> **Lessons for agent builders.** (1) Spend the LLM on choosing, not scoring: one 1k-token call over a shuffled path list beat three minutes of likelihood scoring on the same files. (2) Buy recall before ranking: a 90 MB embedder's top-25 lifted the pool ceiling from 63% to 89%. (3) Shuffle candidates with a fixed seed and report the pool ceiling. (4) Before trusting a public-benchmark number, run an empty-issue control and a post-cutoff slice — both cost minutes. (5) Gate evaluation harnesses on the official fix: ours caught silent environment failures on the first instance.

5. Reproducibility and disclosure

Code, data and a CPU notebook recomputing every number: github.com/HussamHaroon/RecallFirst. Released: per-instance orderings and Gemma answers, the 57 and 120 instance IDs, control and probe option lists, the scorer, statistics and figure scripts, the patch harness and an engineering log. Seeds: candidate shuffle "`<instance>|listwise`", bootstrap 20261107. Re-running the (pre-fix) scorer on a different A100 VM 13 days later reproduced every likelihood variant's gold-file rank on 24/24 earlier-study instances. We disclose the padding bug, the CodeRankEmbed fallback and two rounds of harness bugs. Every number was recomputed from the released files by an independent analysis pass. Experiments were run with AI-agent assistance under human direction.

6. Limitations

One model (Gemma-4-12B-it), one embedder and Python only; 57 Lite instances in 8 repositories, so per-repository claims are not made. The post-cutoff repositories are smaller than Lite's, so absolute rates are not comparable across the two sets; the paired comparison with dense on the same instances is. The name-free conclusions rest on 23 and 49 instances. The likelihood scorer used the instruction-tuned checkpoint and max-over-chunks aggregation, neither ablated.

7. Conclusion

Asking a frozen model how surprised it is by code localizes bugs worse than BM25. Asking it to choose among good candidates does — on the same candidates, on a higher-recall pool, and on issues written after it was trained. Its edge over a dense retriever is largest when the issue points at the file; closing the gap on the rest is the open problem.

Appendix A. Selector prompt (verbatim)

`{repo}`, `{n}`, `{issue}` and the listing are filled per instance; candidates are shuffled with a fixed per-instance seed. The bracketed clause and the per-file symbol lists appear only in the paths + names condition. The 10-way probe uses the same header, a lettered list and "Which single file is most likely modified to fix this bug? Answer with just the letter."

```
Below is a bug report from the {repo} project, followed by a numbered list of {n}
candidate source files from the repository [(with the classes/functions each file
defines)]. Rank the 10 files most likely to be modified to fix this bug, most likely
first. Answer with only the 10 numbers, comma-separated.

--- BUG REPORT ---
{issue}

--- CANDIDATE FILES ---
1. django/db/models/query.py  [QuerySet, QuerySet.filter, ...]
2. django/utils/autoreload.py  [...]
...

Answer (10 numbers, most likely first):
```

References

[1] S. Zhuang, B. Liu, B. Koopman, G. Zuccon. Open-source LLMs are strong zero-shot query likelihood models for document ranking. Findings of EMNLP 2023. arXiv:2310.13243.

[2] A. Holtzman et al. Surface Form Competition. EMNLP 2021. arXiv:2104.08315.

[3] Liu et al. Pointwise mutual information as a performance gauge for RAG. NAACL 2025. arXiv:2411.07773.

[4] B. Ray et al. On the “naturalness” of buggy code. ICSE 2016.

[5] R.-M. Karampatsis et al. Big Code != Big Vocabulary. ICSE 2020. arXiv:1911.04621.

[6] W. Sun et al. Is ChatGPT good at search? Investigating LLMs as re-ranking agents (RankGPT). EMNLP 2023. arXiv:2304.09542.

[7] Li, Jiang, Huang, Gu. LLM-BL. ICPC 2025.

[8] T. Suresh, R. G. Reddy et al. CoRNStack: High-quality contrastive data for better code retrieval and reranking. ICLR 2025. arXiv:2412.01007.

[9] R. G. Reddy, T. Suresh et al. SweRank: Software issue localization with code ranking. ICLR 2026. arXiv:2505.07849.

[10] C. S. Xia et al. Agentless. arXiv:2407.01489.

[11] Z. Chen et al. LocAgent: Graph-guided LLM agents for code localization (Loc-Bench). ACL 2025. arXiv:2503.09089.

[12] S. Liang, S. Garg, R. Z. Moghaddam. The SWE-Bench Illusion: When state-of-the-art LLMs remember instead of reason. NeurIPS 2025. arXiv:2506.12286.

[13] Does SWE-Bench-Verified test agent ability or model memory? arXiv:2512.10218, 2025.

[14] C. E. Jimenez et al. SWE-bench. ICLR 2024. arXiv:2310.06770.

[15] N. Reimers, I. Gurevych. Sentence-BERT. EMNLP 2019 (checkpoint: sentence-transformers/all-MiniLM-L6-v2).

[16] Gemma Team, Google. Gemma 4 model card and EU training-data summary. 2026.

[17] L. Zhang et al. SWE-bench Goes Live! arXiv:2505.23419, 2025.